SAECOIN

API SPECIFICATION

Document Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Blockchain Type: Independent Layer-1
API Style: REST + P2P Protocol
Document Status: Proposed

---

1. PURPOSE

This document defines the API interfaces used by SAECOIN applications and services.

The API layer will allow communication between:

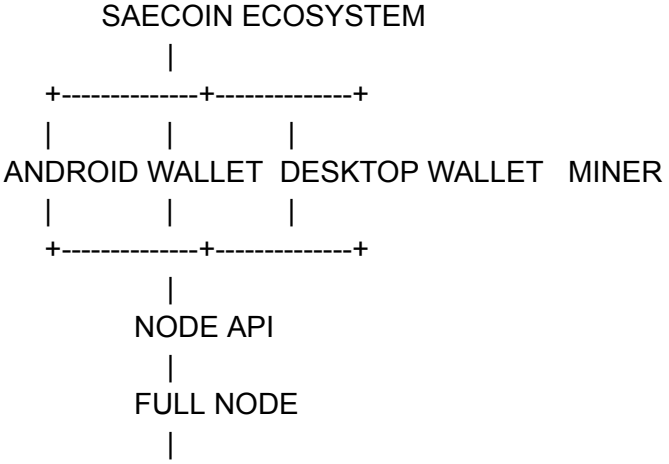
- Android Wallet.
- Windows Wallet.
- Full Node.
- Mining Software.
- Block Explorer.
- Explorer Indexer.
- Mining Pool.
- Network Monitoring Services.

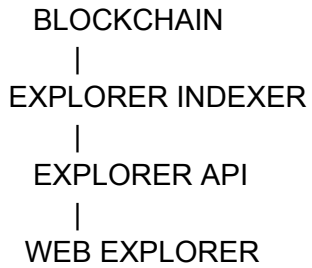
The API must provide consistent, secure, and predictable communication.

---

2. API ARCHITECTURE

Proposed communication structure:





The Node API and Explorer API must remain logically separate.

The Explorer API must not be treated as the authoritative source of consensus state.

---

3. API TECHNOLOGY

Proposed technologies:

Component	Technology
API Backend	Rust
HTTP API	REST
Data Format	JSON
Internal P2P Communication	Custom Versioned Binary Protocol
API Documentation	OpenAPI
API Testing	Automated Integration Tests

JSON may be used for application communication.

Consensus-critical block and transaction serialization must use the deterministic binary format defined by the consensus protocol.

---

4. NETWORK ENVIRONMENTS

All APIs must support separate network environments.

Network	Purpose
Development	Local development
Testnet	Public testing
Mainnet	Production

Each network must have a distinct network identifier.

API responses should identify the network to prevent accidental mixing of testnet and mainnet information.

---

5. API BASE PATH

Proposed base paths:

/api/v1/node
/api/v1/blockchain
/api/v1/transactions
/api/v1/wallet
/api/v1/mining
/api/v1/explorer
/api/v1/network

Local development example:

http://127.0.0.1:<port>/api/v1

Production remote communication must use HTTPS.

---

6. COMMON RESPONSE FORMAT

Successful responses should use a consistent JSON structure.

Example:

```
{
  "success": true,
  "data": {},
  "error": null
}
```

Error response:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "INVALID_REQUEST",
    "message": "The request is invalid."
  }
}
```

Sensitive internal information must never be included in error responses.

---

7. NODE API

The Node API provides information about a running SAECOIN node.

7.1 Node Status

Endpoint

GET /api/v1/node/status

Purpose:

Returns current node status.

Example response:


```
{
  "success": true,
  "data": {
    "network": "development",
    "node_version": "0.1.0",
    "block_height": 0,
    "syncing": false,
    "peer_count": 0,
    "status": "running"
  },
  "error": null
}
```


7.2 Node Information

GET /api/v1/node/info

Returns:

- Node version.
- Network ID.
- Protocol version.
- Software build information.

7.3 Node Peers

GET /api/v1/node/peers

Returns connected peer information.

Sensitive peer metadata should not be exposed unnecessarily.

---

8. BLOCKCHAIN API

8.1 Blockchain Information

GET /api/v1/blockchain/info

Returns:

- Current block height.
- Best block hash.
- Network identifier.
- Synchronization status.
- Cumulative chain work, if exposed.

8.2 Get Block by Hash

GET /api/v1/blockchain/block/{hash}

Returns block information.

8.3 Get Block by Height

GET /api/v1/blockchain/block/height/{height}

Returns the block at the requested height on the active chain.

8.4 Get Latest Blocks

GET /api/v1/blockchain/blocks/latest?limit=20

Returns recent active-chain blocks.

The API must enforce a maximum page size.

---

9. TRANSACTION API

9.1 Get Transaction

GET /api/v1/transactions/{txid}

Returns:

- Transaction ID.
- Inputs.

- Outputs.
- Transaction status.
- Block information, if confirmed.
- Confirmation count.

9.2 Broadcast Transaction

POST /api/v1/transactions/broadcast

Example request:

```
{
  "network": "development",
  "raw_transaction": "HEX_ENCODED_TRANSACTION"
}
```

Example response:

```
{
  "success": true,
  "data": {
    "transaction_id": "TRANSACTION_HASH",
    "status": "accepted_to_mempool"
  },
  "error": null
}
```

The node must independently validate the transaction before accepting it into the mempool.

A successful broadcast response does not mean the transaction is confirmed.

---

10. UTXO API

10.1 Get Address UTXOs

GET /api/v1/wallet/utxos/{address}

Returns available UTXOs associated with the address.

Example:

```
{
  "success": true,
  "data": {
    "address": "SAECOIN_ADDRESS",
    "utxos": [
```

```
      {
        "txid": "TRANSACTION_HASH",
        "output_index": 0,
        "amount": 100000000,
        "confirmed": true
      }
    ]
  },
  "error": null
}
```


Amounts are expressed in smallest SAC units.

The API must distinguish confirmed UTXOs from any optional unconfirmed wallet information.

---

11. WALLET API

Wallet APIs support wallet-related application operations.

Private-key operations must remain inside the wallet security boundary.

11.1 Get Wallet Balance

GET /api/v1/wallet/balance/{address}

Returns the balance calculated from relevant UTXOs.

11.2 Get Address Transactions

GET /api/v1/wallet/transactions/{address}

Returns transaction history available to the node or wallet index.

11.3 Create Transaction

Transaction creation and signing should normally be performed by the wallet core locally.

The node may provide UTXO and fee information but must not request the user's seed phrase or private key.

11.4 Address Validation

POST /api/v1/wallet/validate-address

Example request:

```
{
  "address": "SAECOIN_ADDRESS"
}
```

Example response:

```
{
  "success": true,
  "data": {
    "valid": true,
    "network": "development"
  },
  "error": null
}
```

---

12. MINING API

The mining API allows compatible mining software to communicate with a node.

12.1 Mining Status

GET /api/v1/mining/status

Returns:

- Mining enabled status.
- Current block height.
- Current difficulty.
- Target block time.
- Connected mining workers, if applicable.

12.2 Get Mining Job

GET /api/v1/mining/job

Returns candidate block information according to the mining protocol.

12.3 Submit Mining Solution

POST /api/v1/mining/submit

Example request:

```
{
  "job_id": "JOB_IDENTIFIER",
```

```
  "nonce": "MINING_NONCE",
  "solution": "PROOF_DATA"
}
```

The exact request format depends on the final PoW algorithm.

The node must independently validate every submitted solution and block.

---

13. EXPLORER API

The Explorer API provides indexed blockchain information.

13.1 Latest Blocks

GET /api/v1/explorer/blocks

13.2 Block Details

GET /api/v1/explorer/block/{hash}

13.3 Transaction Details

GET /api/v1/explorer/transaction/{txid}

13.4 Address Details

GET /api/v1/explorer/address/{address}

13.5 Network Statistics

GET /api/v1/explorer/stats

Possible response fields:

- Block height.
- Estimated network hashrate.
- Current difficulty.
- Total issued supply.
- Latest block time.
- Transaction statistics.

Estimated hashrate must be clearly identified as an estimate.

Explorer statistics must be derived from validated blockchain data.

---

14. NETWORK API

14.1 Network Information

GET /api/v1/network/info

Returns:

- Network name.
- Network ID.
- Protocol version.
- Genesis block hash.
- Current block height.

14.2 Network Peers

GET /api/v1/network/peers

Returns public peer information permitted by node configuration.

---

15. MINING POOL API

Mining pool support may be introduced after solo mining works reliably.

Possible endpoints:

POST /api/v1/pool/register
GET /api/v1/pool/workers
GET /api/v1/pool/statistics
GET /api/v1/pool/payouts
POST /api/v1/pool/submit-share

Pool accounts, worker credentials, shares, and payout records must be managed separately from blockchain consensus.

Pool software must independently verify submitted shares.

---

16. ERROR CODES

Proposed error codes:

Error Code| Meaning
INVALID_REQUEST| Invalid request

INVALID_ADDRESS| Invalid address
INVALID_TRANSACTION| Invalid transaction
INVALID_SIGNATURE| Invalid signature
INSUFFICIENT_FUNDS| Insufficient available funds
UTXO_NOT_FOUND| Referenced UTXO not found
DOUBLE_SPEND| Conflicting transaction
INVALID_BLOCK| Invalid block
INVALID_PROOF| Invalid Proof of Work
NETWORK_MISMATCH| Incorrect network
NODE_SYNCING| Node is synchronizing
RATE_LIMITED| Too many requests
INTERNAL_ERROR| Internal server error

Error messages must not expose private implementation details.

---

17. API SECURITY

Requirements:

- Validate all incoming requests.
- Enforce request size limits.
- Apply rate limiting where appropriate.
- Use HTTPS for remote production APIs.
- Protect privileged node operations.
- Restrict administrative endpoints.
- Avoid exposing wallet secrets.
- Validate network identifiers.
- Prevent unsafe file access.
- Log security events without recording sensitive information.

Public read-only APIs should be separated from privileged administrative APIs.

---

18. API VERSIONING

The API must support versioning.

Initial version:

/api/v1/

Future incompatible changes should use a new API version.

Examples:

/api/v2/

Backward compatibility should be maintained where practical.

API changes must be documented.

---

19. API DOCUMENTATION

The project must maintain an OpenAPI specification.

It should document:

- Endpoint paths.
- HTTP methods.
- Request parameters.
- Request bodies.
- Response formats.
- Error codes.
- Authentication requirements.
- Rate limits.

Documentation should be updated alongside implementation.

---

20. API TESTING

Testing must include:

Functional Tests

- Valid requests.
- Invalid requests.
- Missing parameters.
- Correct response formats.

Security Tests

- Unauthorized access.
- Malformed input.
- Oversized requests.
- Rate limiting.
- Sensitive data exposure.

Integration Tests

- Wallet-to-node communication.
- Miner-to-node communication.
- Explorer-to-node data flow.
- Transaction broadcasting.
- Block lookup.

Network Tests

- Development network.
- Testnet.
- Mainnet configuration validation.

---

21. IMPLEMENTATION PRIORITY

Recommended API development order:

1. Node status.
2. Network information.
3. Blockchain information.
4. Block lookup.
5. Transaction lookup.
6. UTXO lookup.
7. Transaction broadcast.
8. Mining status.
9. Mining job interface.
10. Mining solution submission.
11. Explorer API.
12. Pool API.

The API must be implemented incrementally alongside the corresponding blockchain modules.

---

22. FINAL API PRINCIPLES

1. APIs must not bypass consensus validation.
2. Wallet signing must remain local.
3. Private keys must never be requested by node APIs.
4. Monetary amounts must use integer smallest units.
5. Production remote communication must use secure transport.
6. Public APIs must not expose privileged node controls.
7. Explorer information must be identified as indexed data.
8. Every endpoint must have automated tests.
9. API versions must be documented.
10. API success must not be confused with blockchain confirmation.

---

23. FINAL NOTES

This document defines the proposed SAECOIN API architecture.

Exact endpoint behavior, request limits, authentication, mining job formats, and P2P wire messages must be finalized during implementation.

The API specification must remain consistent with:

- PRD.md
- Schema.md
- Blockchain_Architecture.md
- Consensus_Spec.md
- Security_Spec.md

END OF API_Spec.md